

Handout B — Excerpts: Scaling Tic-Tac-Toe to 9×9

Two experiments scaled the 3×3 game to a 9×9 board with a 5-in-a-row win rule. These excerpts are verbatim from the session transcripts. Read B1 and B2 side by side; the final section is the shipped specification's input-handling rules, for comparison with B1's opening prompt.

B1 — Attempt 1, Session 12: requirements as one paragraph

Session 12 — Scale tic tac toe to 9x9 board with five in a row (2026-05-26)

Branch: 5board

User:

I have a tic tac toe game with a standard 3 by 3 board that I want to adapt to a larger game. Scale the game up for a 9 by 9 board where you need to get 5 in a row to win.

Before changing the code, revise the @SPECS.md and @CLAUDE.md files to reflect the adaptations. Picking a move should now be a two step process where a player first picks a column then a row to play in. When prompted, a special board should be printed that shows the numbered options along the rows/columns. After the column is selected, it should indicate it on the display when selecting the row.

Claude:

I'll start by exploring the current codebase to understand what's there before planning the documentation revisions.

Claude:

→ *Agent: Survey Tic-Tac-Toe codebase*

Claude:

Got the lay of the land. The code is a clean 3x3 with `Game/ComputerAI/main.py` plus a `tests/` folder. Before I draft the new spec, I want to nail down three ambiguities.

Claude:

→ *asked the user a question*

Claude:

Writing the plan file now.

Claude:

→ *Write i-have-a-tic-whimsical-petal.md*

Claude:

→ *ToolSearch*

Claude:

→ *presented plan for approval*

Claude:

→ *Skill /grill-me*

User:

Base directory for this skill: [project root]/.claude\skills\grill-me

Interview me relentlessly about every aspect of this plan until we reach a shared understanding.
Walk down each branch of the design tree resolving dependencies between decisions one by one.

If a question can be answered by exploring the codebase, explore the codebase instead.

For each question, provide your recommended answer.

Claude:

Let me walk down the decision tree one branch at a time. I'll recommend an answer for each.

Question 1 of N — Label visibility.

Your original ask said: “*When prompted, a special board should be printed that shows the numbered options along the rows/columns.*” That phrase is ambiguous — does it mean:

- (A) Labels are part of the board, **always shown** during a game (every render, including after the computer plays, between turns, on the post-game screen).
- (B) Labels appear **only when actively prompting a human** for input (column or row). The post-move board, the computer-move board, and the final game-over board render label-free.

This decision changes a lot of the spec text and the `Game.render` signature design.

My recommendation: (A) always shown. Reasons: - 9x9 is much harder to eyeball than 3x3. Even when it isn't *your* turn to type, you want to read the position. “Computer played column 4 row 7” is far easier to verify against a labeled board. - One render mode is simpler than two — `Game.render` just always emits labels, and the optional `selected_column` parameter is the only variant. - (B) would mean `main.py` has to decide per-call whether to ask for labeled or unlabeled rendering, which leaks a UI concern into game-flow code.

The cost of (A) is a bit of visual noise on every board, but on a 9x9 with . for empties that noise is mild.

Which do you want — (A) always-labeled, (B) only-when-prompting, or some hybrid?

User:

A

Claude:

Locked in: labels always visible during a game.

Question 2 — Internal board storage.

The 3x3 game stores `board` as a flat `list[9]` with a numpad→index translation table. For 9x9, two reasonable choices:

- (A) Flat `list[81]`, indexed $(row-1)*9 + (col-1)$. Generalizes the existing pattern.
- (B) 2D list-of-lists, `board[row][col]` with both 0-indexed internally. `board[0][0]` is top-left, `board[8][8]` is bottom-right.

My recommendation: (B) 2D list-of-lists. Reasons: - The 3x3 code needed the flat-list-plus-translation idiom *because* numpad input doesn't map linearly to a grid. With (col, row) input that's already grid-shaped, the translation step disappears — `board[row-1][col-1]` reads directly with no helper. - Win detection on 9x9 scans in 4 directions from each cell. `board[r][c]`, `board[r][c+1]`, `board[r+1][c+1]`, etc. is dramatically clearer than `board[r*9 + c]`, `board[r*9 + c+1]`, `board[(r+1)*9 + (c+1)]`. - Rendering becomes a `for row in board:` loop — also clearer. - CLAUDE.md already calls out board indexing as “easy to get wrong.” Moving to 2D removes a class of off-by-one bugs.

The cost of (B): it's a structural shift from the 3x3 design rather than a continuation. But the 3x3's flat list was a workaround for the numpad layout, and that workaround isn't needed anymore.

Which do you want — (A) flat or (B) 2D?

User:

B

Claude:

Locked in: 2D list-of-lists, 0-indexed internally.

Question 3 — Public API parameter order.

You told the user to enter column first, then row. But the public `Game` API could expose either order:

- (A) `make_move(col, row)` — matches the input order the user types.
- (B) `make_move(row, col)` — matches the internal `board[row][col]` indexing (and matches the mathematical “matrix indexing” convention).

The same choice cascades to `available_moves()` (tuple order) and `random_move()` (tuple order).

My recommendation: (A) `make_move(col, row)`. Reasons: - The user-facing prompt collects column first, so at the call site in `main.py` you naturally have `col` then `row`. `make_move(col, row)` reads as “make the move at the column and row the user just typed.” `make_move(row, col)` requires swapping at every call site. - “Column, row” is also how grid coordinates are most often spoken in games (“E4,” “column 5 row 3”) — column is the horizontal axis, conventionally the first coordinate (like `x, y`). - The 2D indexing concern is local to `Game`’s internals. Inside `Game`, methods can do `self.board[row-1][col-1]` and that complexity stays hidden.

The cost: a small mental “flip” at the one place inside `Game` where parameters meet the 2D array. `CLAUDE.md` will need to flag this explicitly as the new “easy to get wrong” — the public API is `(col, row)` but the internal indexing is `[row][col]`.

Which do you want — (A) `(col, row)` or (B) `(row, col)`?

User:

Lets use option B, but also change the selection order so we select the row first (to stay in convention with the 2d array indexing)

Claude:

Locked in: `(row, col)` everywhere, and the human picks **row first, then column**.

That swap turns “indicate the chosen column with a `v` caret above the column header” into the opposite problem: indicate the chosen row when prompting for the column. The earlier preview you picked doesn’t apply anymore.

Question 4 — Selected-row indicator.

Options:

- (A) A `>` caret in the left margin, on the line of the chosen row, pointing into the board:

```
  1 2 3 4 5 6 7 8 9
+-----+
1 | . . . . . . . . |
2 | . . . . . . . . |
3 | . . . . . . . . |
> 4 | . . . . . . . . |
5 | . . . . . . . . |
...
Row 4 chosen. Enter a column (1-9):
>
```

- (B) Brackets around the chosen row's label:

```

      1 2 3 4 5 6 7 8 9
      +-----+
1 | . . . . . . . . |
2 | . . . . . . . . |
3 | . . . . . . . . |
[4]| . . . . . . . . |
5 | . . . . . . . . |
...

```

- (C) Text-only: no decoration on the board itself; the prompt line states **Row 4 chosen. Enter a column (1-9):.**

My recommendation: (A) > caret in the left margin. Reasons: - Symmetric to the v caret you originally picked for columns — same visual idea, rotated 90° for the row axis. - Doesn't shift any character widths on the board's grid line, so columns still line up under the header. Brackets in (B) push the | separator one character right on that single row, which breaks alignment. - More visible than text-only (C) when the user re-checks the board mid-prompt.

The caret in option (A) lives in the column reserved for row labels — it temporarily *replaces* the visual gap before the row number, so the actual row label still appears on its line. (Or, equivalently: row labels normally have a leading space; for the chosen row that space becomes >.)

Which do you want — (A) > in the left margin, (B) brackets, or (C) text-only?

User:

A, but be sure to always have the extra two characters of space in front of every row. We don't want any visual misalignment

Claude:

Locked in: > caret in a **fixed 2-character left margin** that every row reserves whether selected or not. The column header line will need matching left padding so the column numbers stay aligned over their cells.

Question 5 — Empty cell glyph.

The original 3x3 spec used a blank space for empty cells (once play had started). The preview you picked during the earlier multi-question round used ., but that was bundled with other choices — worth confirming on its own.

- (A) . (dot) for empty cells.
- (B) Blank space for empty cells.

My recommendation: (A) dot. Reasons: - 9x9 is busy. Dots give every cell a visual anchor — your eye can count “1, 2, 3, 4...” across a row without losing place. With blanks, you're tracking the | separators only, which is harder on a wider board. - It makes diagonals visible. A row of X marks against a row of blanks vs. a row of X marks against a row of dots — the dots make the diagonal axis pop because the dots themselves form a faint diagonal grid. - The 3x3's “blank = empty” choice came from the same era as “show numpad digits on the empty board” — both were trying to keep a tiny board readable. With a labeled 9x9, those constraints are gone.

Cost of (A): the board looks slightly busier. But all that “noise” is information.

Which do you want — (A) dot or (B) blank?

User:

A

Claude:

Locked in: . for empty cells.

Question 6 — Full-row handling (the equivalent of the earlier full-column question, flipped).

You previously told me: if a player picks a full column, reject and re-prompt. Now that **row** is step 1, the same question applies to rows. If the player picks a row where every cell is already occupied, the original logic should flip too — but I want to confirm rather than assume.

- (A) During the row prompt, validate that the chosen row has at least one empty cell. If not, error and re-prompt for a row. (Mirror of the earlier full-column rule.)
- (B) Accept any row 1–9; let the failure surface during the column prompt (every column the user tries will be “occupied” and re-prompt). Simpler code, worse UX.
- (C) Allow backing out of the row selection at the column prompt (e.g., type **b** to return to the row prompt). Adds a new control character.

My recommendation: (A) reject full rows at the row prompt. Reasons: - Symmetric to your earlier choice for full columns. The reasoning is identical: catch dead-end input as early as possible, where the user can fix it cheaply. - This means **Game** gets a new helper — I’d call it **row_has_space(self, row)** returning **True** if the row contains any empty cell. (Was going to be **column_has_space** before the swap.)

Cost of (A): one extra method on **Game**, plus one extra branch in **main.py**’s input loop. Trivial.

Which do you want — (A), (B), or (C)?

User:

A

Claude:

Locked in: row prompt rejects full rows; new helper **Game.row_has_space(self, row)**.

Question 7 — Computer move announcement format.

In 1-player mode after the computer moves, **main.py** prints a line announcing what the computer did before re-rendering the board. The 3x3 version says **Computer (0) plays 9**. Now we have two coordinates and need to choose the wording.

- (A) **Computer (0) plays row 4, column 7**.
- (B) **Computer (0) plays (4, 7)**. — terse, matches the (row, col) tuple convention.
- (C) **Computer (0) plays R4 C7**. — compact label form.

My recommendation: (A) Computer (0) plays row 4, column 7. Reasons: - Matches the verbose, plain-English style of the existing prompts (**Your turn (X). Enter a position (1-9):**). The 3x3 game leans into clear over compact and this stays in that voice. - The user is reading the board labels visually as **row N, column M**, so the announcement echoes the same vocabulary — no mental translation needed. - (B) saves characters but forces the reader to remember which slot is row vs. column. On every game.

Cost of (A): four extra words. Negligible.

Which do you want — (A), (B), or (C)?

User:

A

Claude:

Locked in: **Computer (0) plays row 4, column 7**.

Question 8 — **testing_audit.md** handling.

testing_audit.md is a 67-line audit of the current **3x3** test suite — it identifies weak tests, coverage gaps, and proposed fixes. Once we rewrite **SPECS.md**, almost every finding in there becomes stale (the tests it

audits are about to be replaced wholesale when the code is rewritten in a future turn). My plan currently says “leave it alone.” Three options:

- (A) Leave it untouched. It stays in the repo as a historical artifact of the 3x3 era.
- (B) Add a one-line header note: **Note: this audit covers the 3x3 test suite, superseded by the 9x9 scale-up.** Keeps history but warns the reader.
- (C) Delete it as part of this turn — it’ll be misleading the moment SPECS.md changes.

My recommendation: (B) add a one-line “superseded” header. Reasons: - The audit’s *findings* (which tests were tautological, which gaps existed) lose relevance the moment the tests are rewritten — so leaving it as-is in (A) is a future-confusion trap. - But the audit captured real reasoning about test quality, and the same patterns (tautological assertions, coverage gaps) will likely recur in the 9x9 tests. Keeping the document with a “superseded” stamp preserves that thinking as a reference, without misleading anyone into treating its specific findings as current. - (C) throws away useful prior reasoning. Cheaper to leave a one-line note.

This is the only edit `testing_audit.md` would get this turn — no other changes.

Which do you want — (A) leave alone, (B) add “superseded” header, or (C) delete?

User:

C

Claude:

Locked in: delete `testing_audit.md` as part of this turn.

That’s the last branch worth grilling — I now have enough resolved to commit the final plan. Updating it to reflect all eight locked decisions, then I’ll exit plan mode.

Claude:

→ *Write i-have-a-tic-whimsical-petal.md*

Claude:

→ *presented plan for approval*

Claude:

Plan approved. Executing now: rewrite `SPECS.md`, then `CLAUDE.md`, then delete `testing_audit.md`.

Claude:

→ *Write SPECS.md*

Claude:

Now `CLAUDE.md`.

Claude:

→ *Write CLAUDE.md*

Claude:

Now delete `testing_audit.md`.

Claude:

→ *Bash: Remove-Item [project root]/testing_audit.md*

Claude:

SPECS.md and CLAUDE.md are rewritten. The `testing_audit.md` deletion was blocked — there's a user-level PowerShell deny rule on this system, and the auto-mode classifier also flagged it as a destructive action on a pre-existing file without my having explicit per-action approval.

Two options for finishing the job:

1. **You delete it manually** from a terminal: `Remove-Item [project root]/testing_audit.md`
2. **I leave the file in place** but add a one-line “superseded — applies to the 3x3 test suite, no longer current” header to it. This was option (B) from the grilling.

If you don't act on this, the doc just stays where it is and the spec rewrite is otherwise complete. Which do you want?

User:

using @SPECS.md, finish developing the project by changing the code files and developing tests for 100% branch coverage. Use /grill-me if you are unsure on any part of the implementation.

When Developing the tests, focus on implementing the desired behavior laid out in @SPECS.md instead of what is already implemented in the code. If the tests fail, or the tests fail to achieve 100% branch coverage, iterate in the development and try again.

Claude:

Re-entering plan mode. The previous plan was about rewriting docs; this task is about implementing the code + tests against the (now-finalized) spec. Different scope, so I'll overwrite the plan file. First let me read what's currently in the code to understand the shape I'm replacing.

Claude:

→ *Read game.py*

Claude:

→ *Read computer_ai.py*

Claude:

→ *Read main.py*

Claude:

→ *Read test_game.py*

Claude:

→ *Read test_main.py*

Claude:

→ *Read test_computer_ai.py*

User:

[Request interrupted by user]

B2a — Attempt 2, Session 14: “sub-par SPECS” — elicitation before code
Session 14 — Rewrite tic-tac-toe specs for 9x9 and 5-in-a-row (2026-05-27)

Branch: advanced

User:

User invoked `/clear`

User:

This is a 3x3 tic tac toe game. @SPECS.md gives a good intro to what is developed, but it is sub-par for what a SPECS file should contain.

I want to change the board to a 9x9 board where you need to get 5 in a row to win. Help me fully describe the specs of the program by re-writing @SPECS.md. Mention testing proper procedure as well. I want 100% branch coverage.

Claude:

I'll plan a rewrite of SPECS.md to cover the 9x9, 5-in-a-row redesign. Let me start by exploring the current codebase so I understand what the SPECS needs to describe.

User:

[Request interrupted by user]

User:

This is a 3x3 tic tac toe game. @SPECS.md gives a good intro to what is developed, but it is sub-par for what a SPECS file should contain.

I want to change the board to a 9x9 board where you need to get 5 in a row to win. Help me fully describe the specs of the program by re-writing @SPECS.md. Mention testing proper procedure as well. I want 100% branch coverage. Use `/grill-me` to fully flush out the SPECS file so we don't have to change it in the future

Claude:

→ *Skill /grill-me*

User:

Base directory for this skill: `[project root]/.claude/skills/grill-me`

Interview me relentlessly about every aspect of this plan until we reach a shared understanding. Walk down each branch of the design tree resolving dependencies between decisions one by one.

If a question can be answered by exploring the codebase, explore the codebase instead.

For each question, provide your recommended answer.

ARGUMENTS: Redesigning the Tic-Tac-Toe game from 3x3 to 9x9 with 5-in-a-row win condition. Need to fully flesh out a new SPECS.md that covers all design decisions, behaviors, edge cases, and testing requirements (100% branch coverage). Grill me to surface every ambiguity before writing the spec.

Claude:

Let me first explore the codebase so my questions are grounded in what actually exists.

Claude:

__→ Glob `**/*.py`__

Claude:

__→ Glob `**/*.md`__

Claude:

→ *Read game.py*

Claude:

→ *Read computer_ai.py*

Claude:

→ *Read main.py*

Claude:

→ *Read conftest.py*

Claude:

→ *Read test_game.py*

Claude:

→ *Read testing_audit.md*

Claude:

Got the lay of the land. The existing code is small, tight, and well-spec'd for 3x3 — but going to 9x9 with 5-in-a-row breaks the numpad input convention, the render format, and the win-detection assumptions. I have a lot of branches to walk. Let me grill you systematically, starting with the biggest decisions.

Round 1: Input scheme

This is the most cascading decision — it drives parsing, validation, render format, error messages, and tests.

The current game uses **numpad layout** (1-9 maps to a 3x3 grid bottom-up). That doesn't extend to 9x9. We need a new scheme.

My recommendation: row col as two digits separated by space (e.g. 3 5 = row 3, col 5), with rows numbered **1-9 top-down** (matrix/spreadsheet convention) and columns numbered **1-9 left-to-right**. The board renders with row/col header digits as labels.

Why I prefer this over alternatives: - Single index 1-81 is opaque — players can't visually parse positions - Chess-style A1-I9 mixes letter+digit and adds a parsing axis - Numpad-style bottom-up was fine for 3x3 muscle memory but is alien for 9x9 - Row-then-col matches how the board is read

Claude:

→ *asked the user a question*

Claude:

Got it — drop the space. So input is **two adjacent digits like 35** = row 3, col 5. Cleaner. Both axes are 1-9 so there's no parse ambiguity. Stripping the space also means a 2-char canonical input format which makes validation trivial.

Round 2: Render format

A 9x9 board needs labels — counting cells across a 9-wide row is tedious. Also, the old 3x3 had a “show numpad digits as hints when the board is empty, then hide them” rule. With permanent row/col labels, that hint rule becomes redundant and should go away.

My recommendation: always-visible row/col labels. Simple ASCII. Single-space cell content (X / 0 /). Vertical | between cells. Horizontal dashes between rows.

Claude:

→ asked the user a question

Claude:

Good. I'll spec that: - Empty cells render as . always (no equivalent of the old empty-board hint rule — labels make it redundant) - Top and bottom +-----+ borders - Left/right | edges - Cells separated by a single space

I'll confirm the exact format in the final spec mockup.

Round 3: Win condition — overlines

The phrase “5 in a row” hides a classic Gomoku ambiguity: does **6+ in a row** count as a win?

- **Standard Gomoku (Japanese rules):** only **exactly 5** wins. An overline (6 or more consecutive) does *not* count. This favors strategic play.
- **Free-style / “Connect 5”:** **5 or more** wins. Whoever first completes any run of length 5 wins.

My recommendation: 5-or-more wins (free-style). Reasoning: - Simpler to implement and reason about - Simpler to test (no special “this 5-line is overlined” branch to cover) - Matches the colloquial meaning of “5 in a row” - In practice on 9x9 with manual play, overlines are rare anyway

Claude:

→ asked the user a question

Claude:

5+ wins it is.

Round 4: Computer AI on 9x9

The existing `ComputerAI.random_move` is the only strategy. On 3x3 it's a passable opponent. **On 9x9 with 5-in-a-row, random_move is essentially unwinnable for the computer** — a random scatter across 81 cells will essentially never build a 5-line, so the human wins every game unprompted.

This isn't necessarily a problem (you said you wanted a board redesign, not an AI redesign), but it's worth nailing down so the spec is honest about it.

My recommendation: Keep `random_move` as the only initial strategy. Document in §8 (“Future Extensions”) that smarter AI is *obvious* future work given the board size. Keep the pluggable static-method interface unchanged — that's already designed for adding strategies.

Claude:

→ asked the user a question

Claude:

Random only. Smarter AI deferred to §8.

Round 5: Game public API — how methods accept positions

[...the grilling continues for many more rounds before any code is written; excerpt ends here...]

B2b — Attempt 2, Session 16: an implementation-independent ConOps, the same way

Session 16 — Build Concept of Operations document (2026-06-01)

Note: like prior sessions, this capture ends at the moment the transcript was written, so it does not include the final verification of its own append.

User:

Ammend this converstation to the @claude__transcript.md file.

Claude:

→ inspected the existing claude__transcript.md format (TOC, per-session anchors, condensed dialogue style) to plan appending a new session.

User:

I want to make a concept of operations for this project. Use @SPECS.md and /grill-me to complete this task

Claude:

The request redirected to a larger task: author a Concept of Operations (ConOps) for the project. I loaded the /grill-me skill and read SPECS.md in full, then interviewed the user down the design tree.

- *→ Skill: grill-me; Read SPECS.md*

Claude:

→ asked (round 1): purpose/audience, document structure, relationship to SPECS, file location.

User:

Learning artifact (formal) · IEEE/AIAA ConOps sections · “SPECS.md is meant to provide useful information that might need to be re-stated in the ConOps. Do not reference SPECS in the ConOps as ConOps is implementation independent.” · New CONOPS.md at root

Claude:

Key reframe captured: the ConOps must be **implementation-independent and self-contained** — restate operational facts in plain language, never cite SPECS or code-level detail.

→ asked (round 2): current-system baseline for the change justification, how far implementation-independence should go re: interaction style, which operational scenarios to narrate, and which actors to define.

User:

Standard 3×3 tic-tac-toe baseline · Keep user-facing interaction · Scenarios: solo-vs-computer (win), two-player hot-seat, tie, invalid-input recovery · Actors: all three (solo player, hot-seat pair, computer opponent)

Claude:

→ asked (round 3): include a future-capabilities/vision section; depth of user-observable operational policies; whether to include the IEEE analysis section (benefits/limitations/alternatives); supporting matter (glossary / references).

User:

Yes, include vision · Include all observable policies · Full analysis · Glossary only (omit Referenced Documents)

Claude:

All design-tree branches resolved. I wrote the plan and presented it for approval.

- → *Write plan file; ExitPlanMode (approved)*

Claude:

Implemented both deliverables.

- → *Wrote CONOPS.md — a 9-section IEEE-1362-style ConOps (Scope, Current Situation, Justification, Concept for the Proposed System with policies/modes/actors/environment, Operational Scenarios, Impacts, Analysis, Future Capabilities, Glossary), fully implementation-independent.*
 - → *Grep CONOPS.md for implementation terms (SPECS/Python/class/etc.) — zero matches, confirming independence.*
 - → *Edited claude_transcript.md: bumped the session count to 16, added the Session 16 TOC link, and appended this block.*
-

The shipped specification: input handling (for question 4)

From the final 9×9 SPECS.md:

5. Input Handling

- All user input is read with `input()` and stripped of whitespace.
- A single move now requires **two reads**: a row digit, then a column digit. Each is a single digit 1–9.
- Menu input must be a single digit corresponding to a listed option.
- Invalid input prints a short error message and re-prompts **in the current sub-step**. Examples:
 - Non-digit or out-of-range at the row prompt → re-prompt for the row.
 - A row whose every cell is occupied → re-prompt for the row.
 - Non-digit or out-of-range at the column prompt → re-prompt for the column.
 - A column that selects an already-occupied cell → re-prompt for the column.
- The row choice **sticks** once accepted. There is no back-out from the column prompt to the row prompt within the same turn.